

第4回 PCの作業場所を理解し、Codexで編集する

OS・file・folder・権限から、修正と確認へ

2026年10月13日（火）3限 金融論

今日作るもの

HW01で作った `preview_test.md` を `preview_work.md` という名前でコピーし、見出しと表を追加します。変更箇所はdiffで、文書の表示はpreviewで確認します。最後に、作業規則と今回の記録を残します。

作業場所は自分の `Documents/金融論/HW01` です。`preview_test.md` がまだない場合は、HW01の手順書と同じく次のように頼むと作れます。

`preview_test.md`を作って。タイトルを「Codex設定テスト」とし、CodexのModel、Effort、Speedを箇条書きで書いて。

完成時には、次のfileが並びます。

```
HW01/
├─ preview_test.md      HW01で作った原本
├─ preview_work.md      今回編集するコピー
├─ AGENTS.md            作業規則
├─ AI_LOG.md            作業の記録
└─ output/
    └─ rule_check.md     規則に沿って保存した文書
```

1 PCとOSの基本

OSとapplication

OS（Operating System）は、fileの保存、programの実行、機器や権限の管理を担います。applicationは、OSの上で動くsoftwareです。Word、browser、VS Code、Codex appなどが該当します。

OS	主に使うPC・環境	fileを見るapplication
Windows	さまざまなメーカーのPC	Explorer（エクスプローラー）
macOS	AppleのMac	Finder
Linux	serverや計算環境など	環境によって異なります

この授業ではWindowsとMacを扱います。どちらでも文書を保存・編集できますが、保存場所の書き方や操作の名前が異なります。

CPU・memory・SSD・GPU

部品	主な役割
CPU	programの命令を処理し、計算します
memory (RAM)	実行中のprogramやdataを一時的に置きます
SSD	OS、application、fileを保存します。電源を切っても内容が残ります
GPU	画像表示や、多数の計算を並列に行う処理を担います

memoryは作業中の机、SSDは保存用の書庫に相当します。文書を開くと必要な内容がmemoryに読み込まれ、保存すると変更がSSDなどの保存装置に書き込まれます。

2 fileはどこにあるか

file・folder・拡張子

fileは文書、画像、dataなどを保存する単位です。folderはfileや別のfolderをまとめる場所です。directory（ディレクトリ）も、folderとほぼ同じ意味で使います。

file名の末尾にある `.md` や `.pdf` は拡張子（extension）です。fileの形式を見分ける手がかりになります。

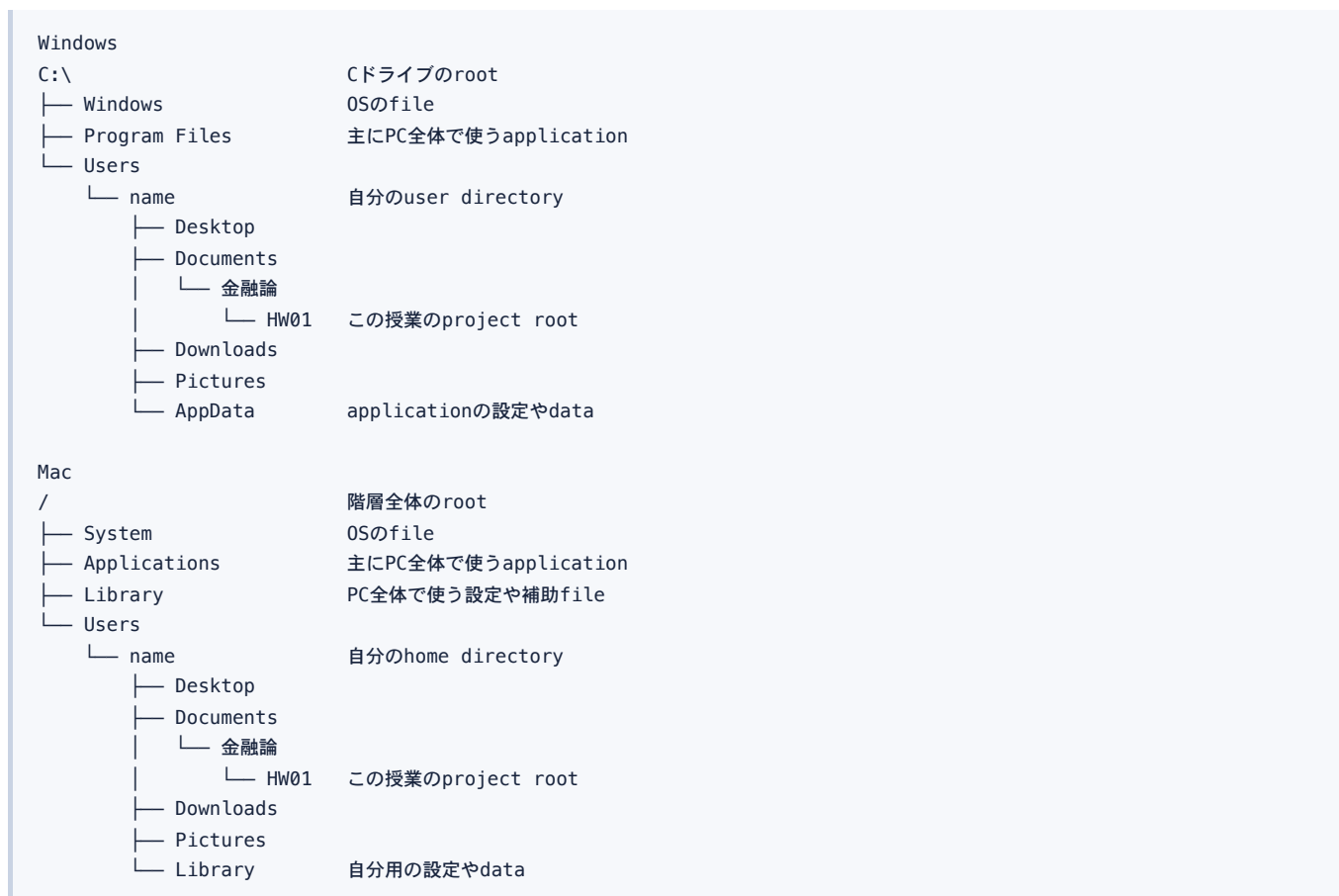
拡張子	内容
<code>.md</code>	Markdownの原稿
<code>.pdf</code>	ページの配置が固定された文書
<code>.docx</code>	Word文書
<code>.csv</code>	行と列からなるdata
<code>.py</code>	Pythonのprogram
<code>.html</code>	browserで表示する文書

FinderやExplorerで拡張子を表示すると、`report.md` と `report.md.txt` を区別できます。名前だけを変えても、中身の形式が変換されるわけではありません。

rootとユーザーフォルダ

root（ルート）は、folderの階層の最上位です。Windowsではドライブごとにrootがあり、Cドライブでは `C:\` です。Macでは `/` が階層全体のrootです。

通常、自分のfileはWindowsでは `C:\Users\name`、Macでは `/Users/name` の下に置きます。この自分用の場所をuser directoryと呼びます。Macではhome directory（ホームフォルダ）とも呼びます。`name` は自分のアカウントのfolder名です。



project rootは、作業の起点に決めたfolderです。この授業では **HW01** が該当します。「projectのrootに保存する」とは、**HW01** の直下に保存する意味です。PC全体の最上位に保存する意味ではありません。

rootには、強い権限を持つユーザーという別の意味もあります。root userは第4章で説明します。

Documents ・ Downloads ・ Desktop

folder	主な使い道	日本語の表示例
Documents	自分で作る文書や課題を整理します	ドキュメント／書類
Downloads	browserなどからdownloadしたfileが入ります	ダウンロード
Desktop	desktop画面に置いたfileが入ります	デスクトップ
Pictures	画像や写真ライブラリなどを保存します	ピクチャ

Downloads の配布資料を **Documents** 内の **金融論/HW01** にまとめると、原稿や出力と同じ場所で扱えます。今回は原本を残し、コピーの **preview_work.md** を編集します。

AppData や **Library** は、applicationが設定やdataを保存する場所です。通常の一覧では隠れている場合があります。

path : fileの住所

pathは、fileやfolderがどこにあるかを表します。例えば今回の作業fileは、次のように表せます。

```
Windows: C:\Users\name\Documents\金融論\HW01\preview_work.md
Mac:      /Users/name/Documents/金融論/HW01/preview_work.md
```

rootからの経路を全部書くのがabsolute path（絶対パス）です。あるfolderを基準に書くのがrelative path（相対パス）です。`HW01` を基準にすると、上のfileは `preview_work.md` です。

```
preview_work.md
output/rule_check.md
figures/fig01.png
```

relative pathの基準は使う場面で変わります。Markdown内の画像やlinkは通常その原稿のfolder、terminalからの操作はその時点の作業場所が基準です。課題内の画像やlinkをrelative pathで書くと、folderごと別のPCへ移しても参照を保ちやすくなります。

Macのterminalで使う `~` は、自分のhome directoryを表します。`~/Documents` は通常 `/Users/name/Documents` です。

隠しfileとfolder

MacやLinuxでは、名前が `.` で始まるfileやfolderは、通常の一覧に表示されないことがあります。Windowsでは、先頭の `.` だけで隠れるとは限らず、隠し属性や表示設定によって変わります。

名前	役割
<code>.venv/</code>	Pythonの実行環境と、pipで入れたlibraryをまとめるfolderです。中身は第6回で扱います
<code>.git/</code>	Gitが変更履歴などを管理するfolderです
<code>.env</code>	programが読む設定値を保存するfileです

`report.md` の末尾の `.md` は拡張子です。`.venv` の先頭の `.` は名前の一部です。

cloudと同期されるfolder

OneDriveやiCloud Driveの設定によって、DocumentsやDesktopの保存先が変わります。例えばWindowsでは `C:\Users\name\OneDrive\Documents` になる場合があります。画面に「ドキュメント」「書類」と表示されても、pathでは `Documents` になる場合があります。

実際の場所はExplorerのアドレス欄やFinderのパスバーで確認できます。「オンラインのみ」のfileは、PCへdownloadしてから使います。cloud同期では上書きや削除も同期されるため、変更前の原本を別に残すことが役立ちます。

3 同じfolderを開いて操作する

GUIとCLI

GUIは、iconやmenuを使って操作する入口です。FinderやExplorerでfolderを開く操作が該当します。CLIは、文字のcommandで操作する入口です。

terminalはcommandを入力する画面です。入力を解釈するprogramをshellと呼びます。Macではzsh、WindowsではPowerShellなどを使います。

GUIとCLIのどちらから操作しても、対象が同じなら同じfileを扱っています。

VS CodeでHW01を開く

VS Codeは、原稿やprogramを編集するapplicationです。「フォルダーを開く」から **HW01** を選ぶと、その中のfileが左側に並びます。

場所	できること
Explorer	folder内のfileを探します
Editor	fileの中身を読み、編集します
Preview	Markdownの見出しや表の表示を見ます
Terminal	commandを実行します

Explorerに **preview_test.md** とそのコピーの **preview_work.md** が並んでいれば、実習の準備ができています。

コラム：VS Codeの仲間

VS Codeの中身は公開されていて、それをもとに作られたeditorがいくつかあります。見た目や組み込まれたAIは違っても、fileとfolderの扱い、Explorer・Editor・Terminalの並びは同じです。

- Cursor。VS Codeを改造して、AIによるcodeの作成と修正を最初から組み込んだeditorです。動かすAIは、OpenAI、Anthropic、Google、xAIのGrokなど複数の会社から選べます。有料の契約が前提です。この授業のVS CodeにCodexの拡張機能を入れた形と、できることはほぼ同じです。
- Antigravity。GoogleがVS Codeをもとに作ったeditorで、Geminiが作業を進めます。Cursorと同じ発想で、動かすAIが違います。
- Positron。RStudioを作っているPositが、VS Codeの基盤の上に作ったeditorです。画面の並びがRStudioに近く、PythonとRの両方で、1行ずつ送って結果を見る使い方が標準です。第14回以降のRの回で、RStudioの代わりに使えます。

どれを使っても、HW01のfolderをそのまま開けます。

current directory：いまの作業場所

current directoryは、terminalでcommandを実行するときの基準となるfolderです。VS Codeの「ターミナル」から新しいterminalを開き、次を入力すると現在地が表示されます。

```
pwd
```

表示が自分の **HW01** なら、続けて次を入力すると、その中のfileやfolderが表示されます。

```
ls
```

MacのzshとWindows PowerShellでは、この二つを同じ名前で使えます。FinderやExplorerの一覧と比べると、GUIとCLIが同じ場所を見ていることを確認できます。

別の場所が表示された場合は、`HW01` を開いているwindowでterminalを開き直すか、表示されたpathをCodexに伝えと、移動方法を確認できます。homeは自分のfileの起点、current directoryはその時点の作業場所です。

4 誰の権限で操作するか

一般ユーザー・管理者・root user

権限は、どのfileを読めるか、変更できるか、どの設定を変えられるかを決めます。administratorは管理者という意味です。WindowsにもMacにも管理者として設定されたアカウントがあります。

自分のDocumentsでの原稿編集は、通常の権限で進められます。PC全体の設定変更などには、管理者の権限が必要になることがあります。管理者のアカウントでログインしていても、すべてのapplicationが常に強い権限で動くわけではありません。

MacやLinuxのroot userは、システムの広い範囲へアクセスできる特別なユーザーです。folderの最上位を表すroot directoryや、作業の起点を表すproject rootとは別の意味です。

Macのsuとsudo

`su` は、別のユーザーに切り替えてshellを開くcommandです。切り替えた先のユーザーとして操作を続けます。ユーザーを指定しない場合はrootへの切替を試みます。

`sudo` は、許可されたユーザーが指定したcommandを別のユーザーの権限で実行する仕組みです。通常はrootの権限を使います。Macの標準的な設定では管理者が利用できます。

Macではrootでのログインが標準で無効になっています。管理者は、rootのログインを有効にせずに `sudo` を使えます。原稿を編集する今回の実習では、どちらも使いません。Appleの説明

Windowsの「管理者として実行」

Windowsでは、applicationを「管理者として実行」すると、通常はUAC（ユーザーアカウント制御）の確認画面が出ます。管理者アカウントでは承認を、一般ユーザーでは管理者の認証情報を求められます。組織の設定によっては操作が制限されます。Microsoftの説明

Windows標準のPowerShellには、Macと同じ `su` はありません。Windows 11 version 24H2以降にはWindows版の `sudo` もありますが、利用には有効化が必要です。

管理者権限を使うと、誤った操作の影響も広がります。「アクセスが拒否されました」と出た場合は、保存先のpathと、そのfolderへの書き込み権限を確認すると原因を絞れます。

CodexのPermissions

OSの権限に加えて、Codexには作業できる範囲を決めるPermissionsがあります。sandboxは技術的にアクセスできる範囲、approvalは追加の操作を承認する仕組みです。Codex側で操作を承認しても、OSの管理者になったことにはなりません。

授業では Ask for approval を使います。承認画面が出たときは、操作の内容、対象のfile、外部への接続や書き込みを読みます。意味が分からない場合は、次のように聞くと判断材料を得られます。

この操作が必要な理由、変更されるfile、元に戻す方法を説明して。
まだ実行しないで。

設定の詳細はOpenAIのPermissionsの説明にあります。

5 Codexに修正を依頼し、結果を見る

projectとthread

Codex appで、自分の HW01 folderをprojectとして開きます。この実習ではPC上のfolderで作業するLocalを使います。VS Codeで開いているfolderと同じpathなら、両方から同じfileを扱えます。

threadは一つの会話です。同じLocalのfolderについてthreadを分けても、fileは共通です。会話を増やすだけでは、作業用のコピーは増えません。

対象・作業・範囲・完了条件を伝える

依頼には、何をどう変え、どこまでを対象にし、何を確認するかを書きます。

要素	今回の例
対象	preview_work.md
作業	見出しと表を追加します
範囲	末尾だけを変更します
完了条件	diffを示し、表の表示を確認します

まず次の依頼で、コピーに見出しと表を追加します。

preview_work.mdの末尾だけに、次の見出しと表を追加して。
既存の行と他のfileは変更しないで。
保存後にdiffを表示して。

今日の確認

確認	見るもの	
---	---	
diff	変更行	
preview	完成時の表示	

diffで追加を確認する

diffは、変更前と変更後のtextの差です。Codexの回答に付いた変更fileの表示から開くか、回答内のdiffを読みます。今回の追加は、次のように表示されます。

```
+## 今日の確認
+
+| 確認 | 見るもの |
+|---|---|
+| diff | 変更行 |
+| preview | 完成時の表示 |
```

+ は追加を表し、画面では一般に緑色になります。行頭の + 自体が原稿に書き込まれるわけではありません。今回の変更では、追加先が末尾で、変更されたfileが `preview_work.md` だけであることが確認できます。

diffで置換を確認する

同じthreadで、表の1か所だけを直します。

```
preview_work.mdの表にある「変更行」だけを
「削除された行と追加された行」に置き換えて。
それ以外は変更しないで。保存後に今回のdiffを表示して。
```

期待するdiffは、1行の削除と1行の追加です。

```
-| diff | 変更行 |
+| diff | 削除された行と追加された行 |
```

- は削除を表し、画面では一般に赤色になります。保存後の原稿には新しい行だけが残ります。diffでは、指定した場所以外が変わっていないかも分かります。

previewで完成時の表示を見る

VS Codeで同じ `preview_work.md` を開きます。Macは `Shift + Command + V`、Windowsは `Ctrl + Shift + V` でMarkdownのpreviewを開けます。Command Paletteの `Markdown: Open Preview to the Side` では、原稿とpreviewを並べられます。

previewでは「今日の確認」が見出しになり、2列の表が表示されます。表には見出し行と2行の内容があります。diff欄の内容は「削除された行と追加された行」です。

確認方法	分かること
diff	どの文字や行を変更したか、指定外の変更がないか
preview	見出しや表が読みやすく表示されるか

diffの赤い行・緑の行は、完成した文書には表示されません。変更箇所をdiffで読み、現在の原稿の見た目をpreviewで確かめると、両方の確認ができます。

6 規則と記録を残す

AGENTS.mdとAI_LOG.md

AGENTS.md は、そのprojectで繰り返し使う作業規則です。AI_LOG.md は、今回何を依頼し、何を採用したかを残す記録です。

file	書く内容の例
AGENTS.md	原資料を保護します。生成物は output/ に保存します
AI_LOG.md	表を追加し、1行を修正しました。diffとpreviewで確認しました

作業規則は短くすると、何を守るかが明確になります。今回だけの依頼はthreadへ書き、今後も共通して使う規則を AGENTS.md に残します。

規則を置き、保存結果を確かめる

配布の sample_project/AGENTS.md を HW01 の直下へコピーします。すでに AGENTS.md がある場合は、その内容と配布sampleを比べ、必要な規則を追加します。

HW01 を作業場所にした新しいthreadで、次を依頼します。

このfolderのAGENTS.mdを読み、生成物の保存先と記録方法を短く説明して。
その規則に従って、今回のpreview_work.mdで行った変更を1文にまとめた
rule_check.mdを保存し、AI_LOG.mdに作業を追記して。
既存の原稿や配布資料は変更しないで。

sampleの規則なら、文書は output/rule_check.md に保存されます。Explorerで保存先を確かめ、AI_LOG.md の追記を読むと、規則が実際の作業へ反映されたか分かります。

AGENTS.md はAIへの指示であり、アクセス制限そのものではありません。規則の読み込みの仕組みはOpenAIのAGENTS.mdの説明にあります。

自分で編集した内容をAIへ渡す

VS Codeで手作業した内容は、保存するとdisk上のfileに反映されます。Macは Command + S、Windowsは Ctrl + S で保存できます。

自分の編集を終えて保存してから、Codexに「現在のfileを読み直して」と伝えると、保存した内容をもとに作業が続けられます。一つのfileを人間とAIが同時に書き換えると変更が衝突するため、交代して編集します。

HW01のreportへの応用

同じ流れで、自分のreportの1段落を直せます。対象の段落と参照するPDFを指定すると、確認する範囲が明確になります。

chapter10_report.mdの「マネーストック」の段落だけを、
moc_text_claude.pdfと照合して。
食い違う文があれば、根拠のpageと修正案を示して。
まだfileは変更しないで。

根拠と修正案を読んだ後、「その1文だけ反映して」と依頼します。保存後は、実習と同じようにdiffとpreviewを確認します。

この回の要点

- fileの場所はpathで表します。project rootとcurrent directoryが分かると、作業や保存の基準を確認できます。
- root directoryは場所、root userは権限を持つユーザーです。自分の課題folderの編集は通常の権限で進められます。
- Codexへの依頼は、対象・作業・範囲・完了条件を明確にします。
- 修正はdiffとpreviewで確認し、規則は `AGENTS.md`、作業の記録は `AI_LOG.md` に残します。